Experiment No. – 8						
Date of Performance:	**16/4/2026**					
Date of Submission:	**23/4/2026**					
Conceptual Understanding (Cognitive) (20%) (02)	Code Execution (Psychomotor) (15%) (1.5)	Problem-Solving & Debugging (Cognitive + Psychomotor) (20%) (02)	Professional Conduct & Lab Ethics (Affective) (20%) (02)	Output / Result Accuracy (Product) (15%) (1.5)	Documentation & Result Analysis (Cognitive) (10%) (01)	Sign with Date

Experiment No. 8

1.1 Aim: Design and deploy smart contract for Academic Certificate Verification System:A university wants to issue tamper-proof digital certificates to graduates. Employers should be able to verify authenticity through a public blockchain without contacting the university. Smart Contract Requirements: Only university admin can issue certificates.Store student ID, name, course, year, and certificate hash.Public function for verification of certificate authenticity.Prevent duplicate certificate entries. Handle both batch issuance and single issuance.

1.2 Course Outcome: Develop the application using the concept of Ethereum with smart contracts using different development tools..

1.3 Learning Objectives: Develop a smart contract

1.4 Requirement:

Software Requirements

- Remix IDE
- MetaMask
- Web browser (Chrome/Edge)
- Solidity programming language

Hardware Requirements

- Computer/Laptop
- Stable Internet connection

Functional Requirements

- Only admin (university) can issue certificates
- Store:
    - Student ID
    - Name
    - Course
    - Year
    - Certificate hash
- Public function to verify certificate authenticity
- Prevent duplicate certificate entries
- Support:
    - Single certificate issuance
    - Batch certificate issuance

Blockchain Requirements

- Ethereum network (Sepolia testnet)
- Smart contract deployment using MetaMask

1.5 Related Theory:

1. Blockchain Technology

Blockchain is a decentralized and distributed digital ledger that records transactions across multiple nodes. Each block contains data, a timestamp, and a cryptographic hash of the previous block, making the system secure and tamper-proof. Once data is added, it cannot be altered, ensuring integrity and transparency.

2. Ethereum and Smart Contracts

Ethereum is a blockchain platform that allows developers to build decentralized applications using smart contracts.

A smart contract is a self-executing program stored on the blockchain that automatically enforces rules and conditions without intermediaries.

3. Solidity Programming Language

Smart contracts are written using Solidity, a high-level programming language similar to JavaScript. It is used to define data structures, functions, and access control for blockchain-based applications.

4. Digital Certificates and Verification

Digital certificates are electronic documents issued by institutions to certify achievements. In traditional systems, verification requires contacting the issuing authority.

Using blockchain:

- Certificates are stored securely
- Employers can verify authenticity directly

- Eliminates fraud and manual verification

5. Cryptographic Hashing
A hash function converts input data into a fixed-length string (hash value).
- Each certificate is assigned a unique hash
- Even a small change in data produces a completely different hash
- Ensures data integrity and uniqueness

6. Access Control Mechanism
The system uses an admin-based control:
- Only the university admin can issue certificates
- Implemented using conditions like require() in smart contracts
- Prevents unauthorized access

7. Prevention of Duplicate Entries
The smart contract uses mapping or storage checks to ensure:
- Each certificate hash is unique
- Duplicate certificates cannot be added
- Maintains data consistency

8. Decentralized Verification System
Unlike traditional systems:
- No central authority required during verification
- Anyone can verify using the public blockchain
- Ensures transparency and trust

9. Role of MetaMask and Remix
- MetaMask connects the user to the blockchain
- Remix IDE is used to write, compile, and deploy smart contracts

1.6 Procedure:

Step 1: Setup Environment
1. Install MetaMask in browser
2. Create an account and select Sepolia Test Network
3. Open Remix IDE

Step 2: Create Smart Contract File
1. In Remix → Go to File Explorer
2. Click New File → Name it: Certificate.sol

Step 3: Write Smart Contract Code
Paste the following code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract CertificateVerification {

  address public admin;

  struct Certificate {
     string studentId;
     string name;
     string course;
     uint year;
```

```
    string hash;
  }

  mapping(string => Certificate) public certificates;

  constructor() {
    admin = msg.sender;
  }

  modifier onlyAdmin() {
    require(msg.sender == admin, "Only admin allowed");
    _;
  }

  function issueCertificate(
    string memory _studentId,
    string memory _name,
    string memory _course,
    uint _year,
    string memory _hash
  ) public onlyAdmin {

    require(bytes(certificates[_hash].studentId).length == 0, "Certificate already exists");

    certificates[_hash] = Certificate(_studentId, _name, _course, _year, _hash);
  }

  function verifyCertificate(string memory _hash)
    public view returns (string memory, string memory, string memory, uint)
  {
    Certificate memory cert = certificates[_hash];
    return (cert.studentId, cert.name, cert.course, cert.year);
  }
}
```

Step 4: Compile the Contract

1. Go to Solidity Compiler tab
2. Select version 0.8.x
3. Click Compile Certificate.sol

Step 5: Deploy the Contract

1. Go to Deploy & Run Transactions
2. Select:
    - Environment → Browser Extension (MetaMask)
    - Network → Sepolia Testnet
3. Click Deploy
4. Confirm transaction in MetaMask

Step 6: Issue Certificate (Single Entry)

1. Expand deployed contract
2. Locate issueCertificate()
3. Enter sample data:
    - Student ID → 101
    - Name → Rahul
    - Course → BTech

        - Year → 2025
        - Hash → abc123
    4. Click Transact → Confirm in MetaMask

Step 7: Verify Certificate
    1. Locate verifyCertificate()
    2. Enter hash:
        - abc123
    3. Click Call
    4. Output will display student details

Step 8: Prevent Duplicate Entry Test
- Try issuing again with same hash (abc123)
- Transaction will fail → confirms duplicate prevention

Step 9: Batch Certificate Issuance (Optional)
Add this function in code:

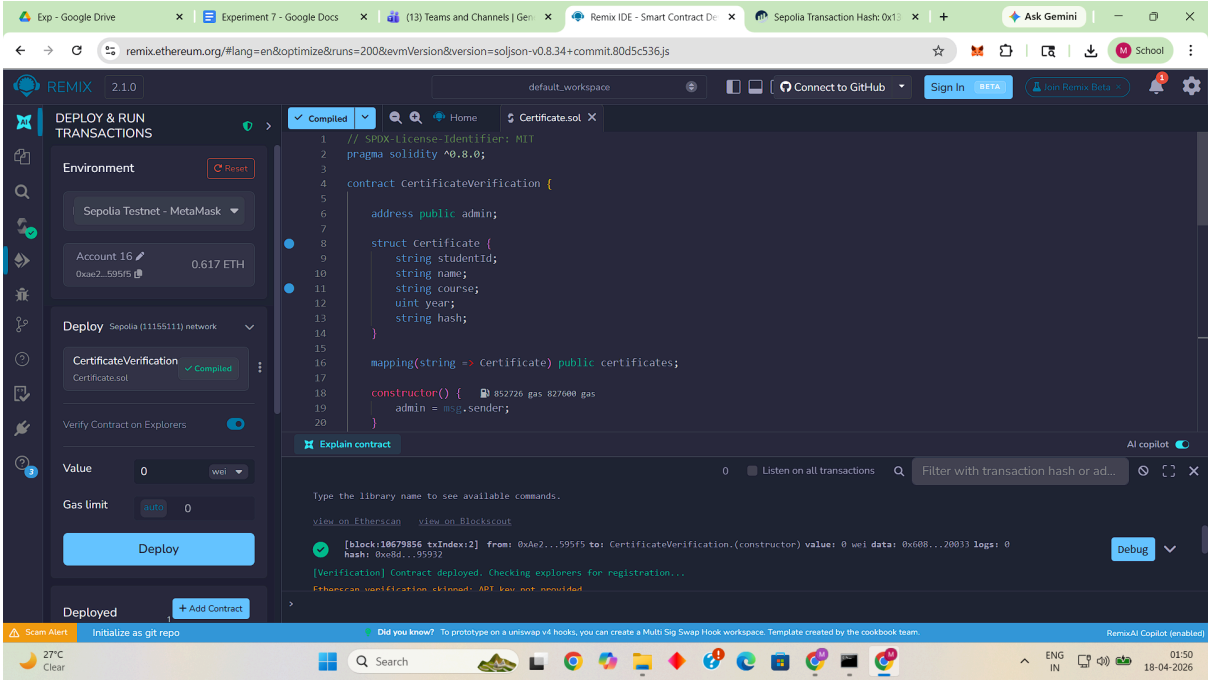
```
function batchIssue(
  string[] memory ids,
  string[] memory names,
  string[] memory courses,
  uint[] memory years,
  string[] memory hashes
) public onlyAdmin {
  for (uint i = 0; i < hashes.length; i++) {
    certificates[hashes[i]] = Certificate(ids[i], names[i], courses[i], years[i], hashes[i]);
  }
}
```

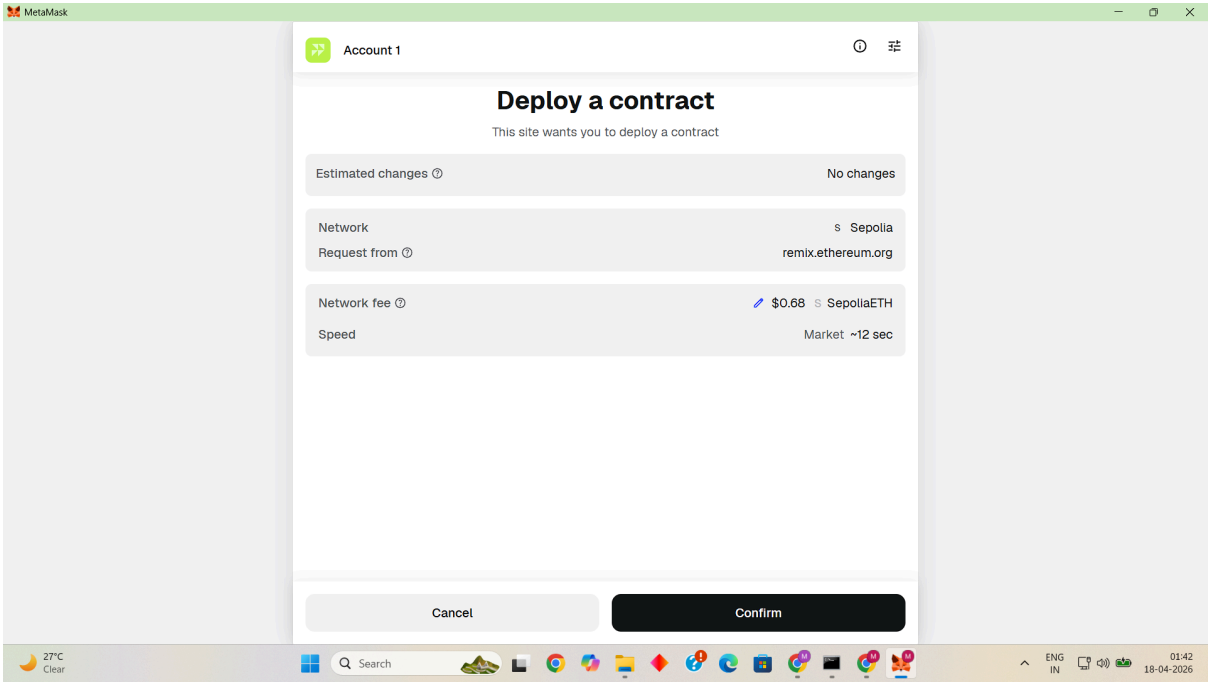
Recompile and deploy again to test batch input.

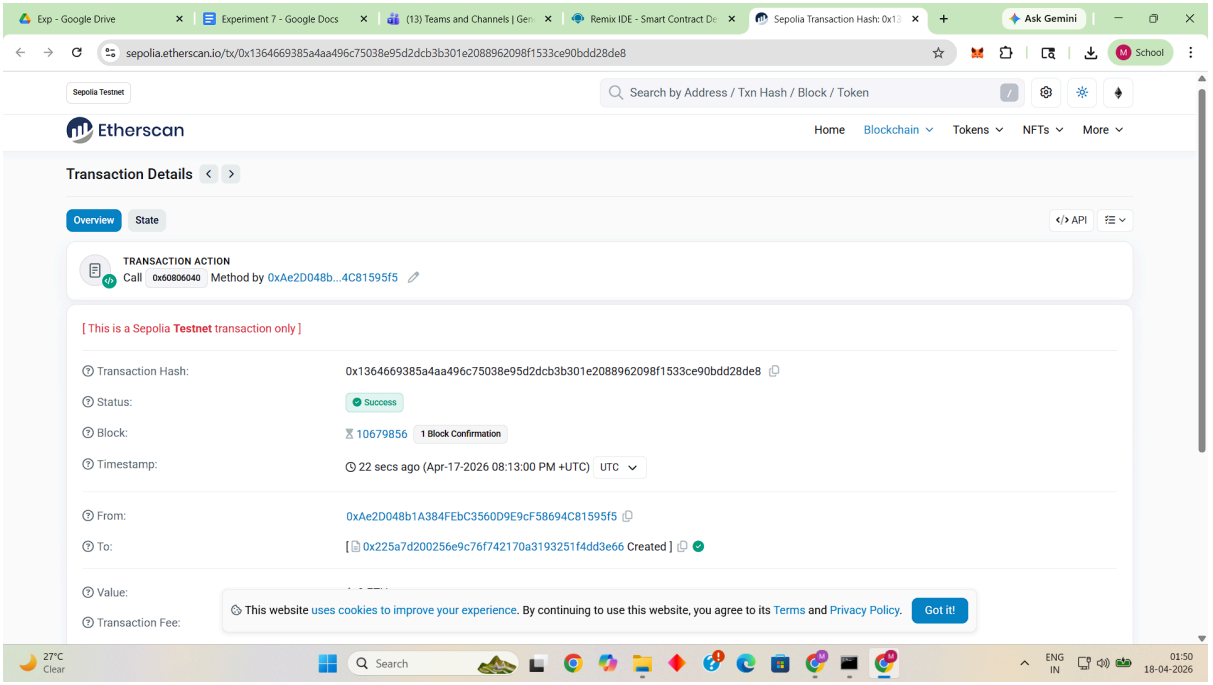
Step 10: Record Results
- Note transaction details
- Observe successful storage and verification
- Take screenshots of:
    - Deployment
    - Issuing certificate
    - Verification output

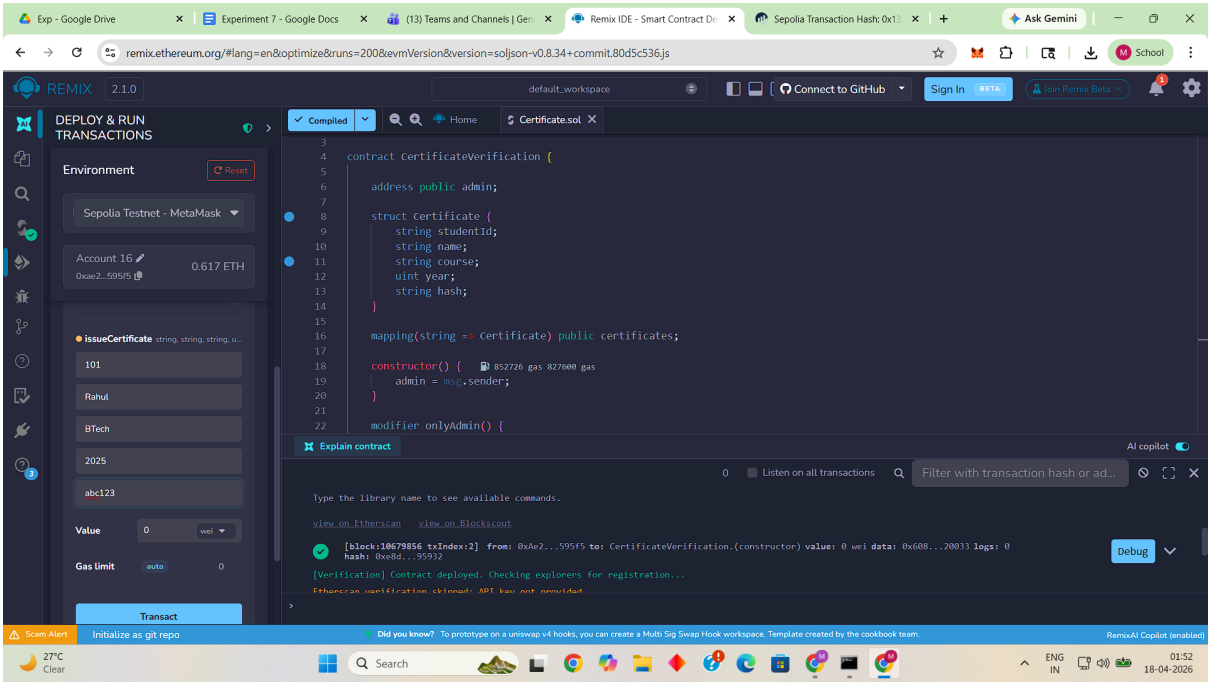
Final Outcome
The smart contract successfully:
- Stores certificate data securely
- Allows public verification
- Prevents duplication
- Ensures only admin access

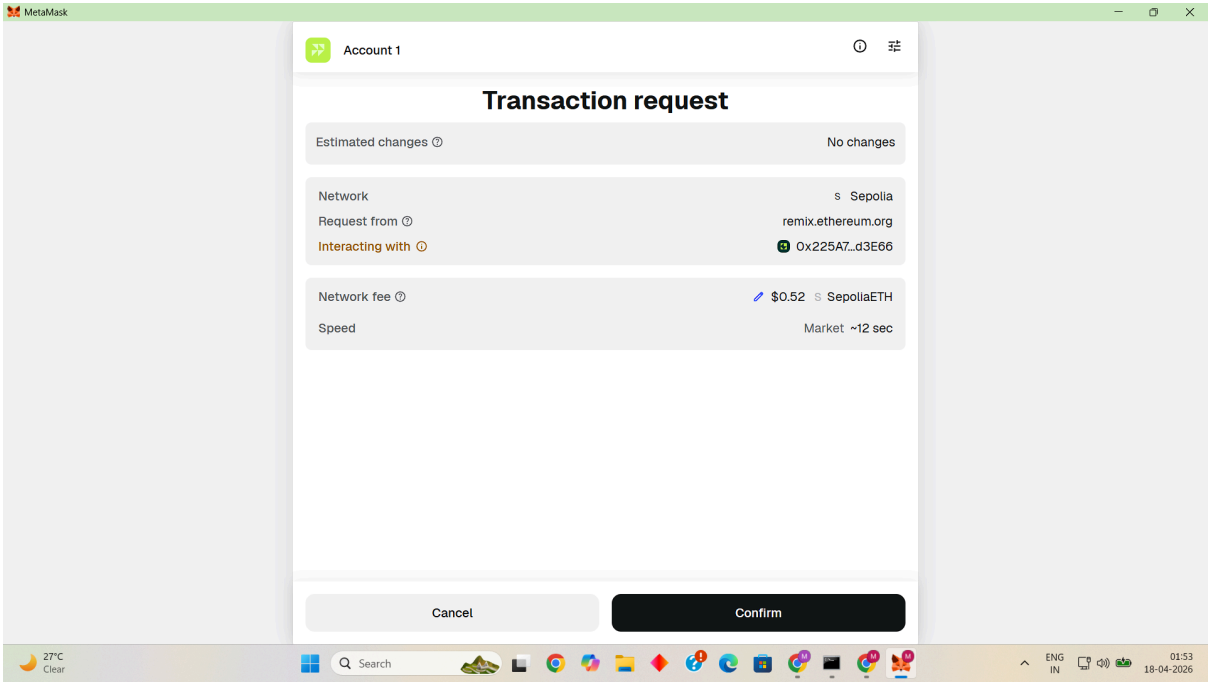
1.7 Program and Output:

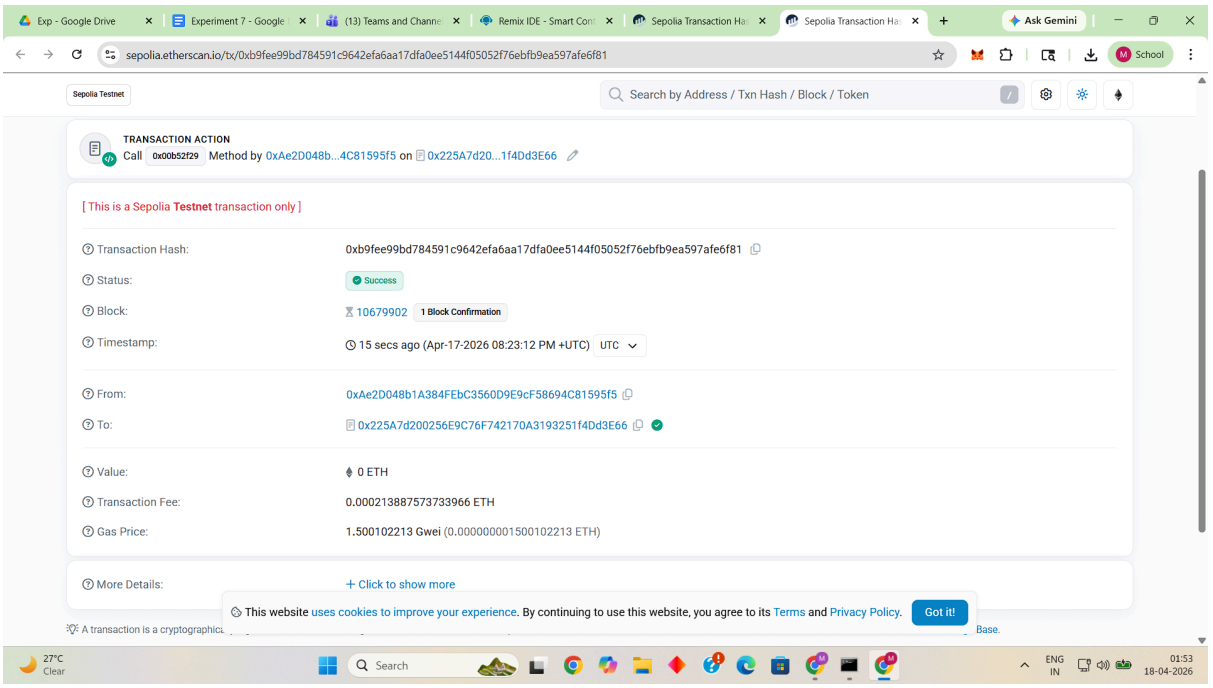

remix.ethereum.org/#lang=en&optimize&runs=200&evmVersion&version=soljson-v0.8.34+commit.80d5c536.js
REMIX 2.1.0
DEPLOY & RUN TRANSACTIONS
Environment
Sepolia Testnet - MetaMask
Account 16
0.617 ETH
Deploy Sepolia (11155111) network
CertificateVerification
Certificate.sol
Verify Contract on Explorers
Value 0 wei
Gas limit 0
Deploy
Deployed
Certificate.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract CertificateVerification {
address public admin;
struct Certificate {
string studentId;
string name;
string course;
uint year;
string hash;
}
mapping(string => Certificate) public certificates;
constructor() {
admin = msg.sender;
}
Explain contract
Type the library name to see available commands.
[Verification] Contract deployed. Checking explorers for registration...

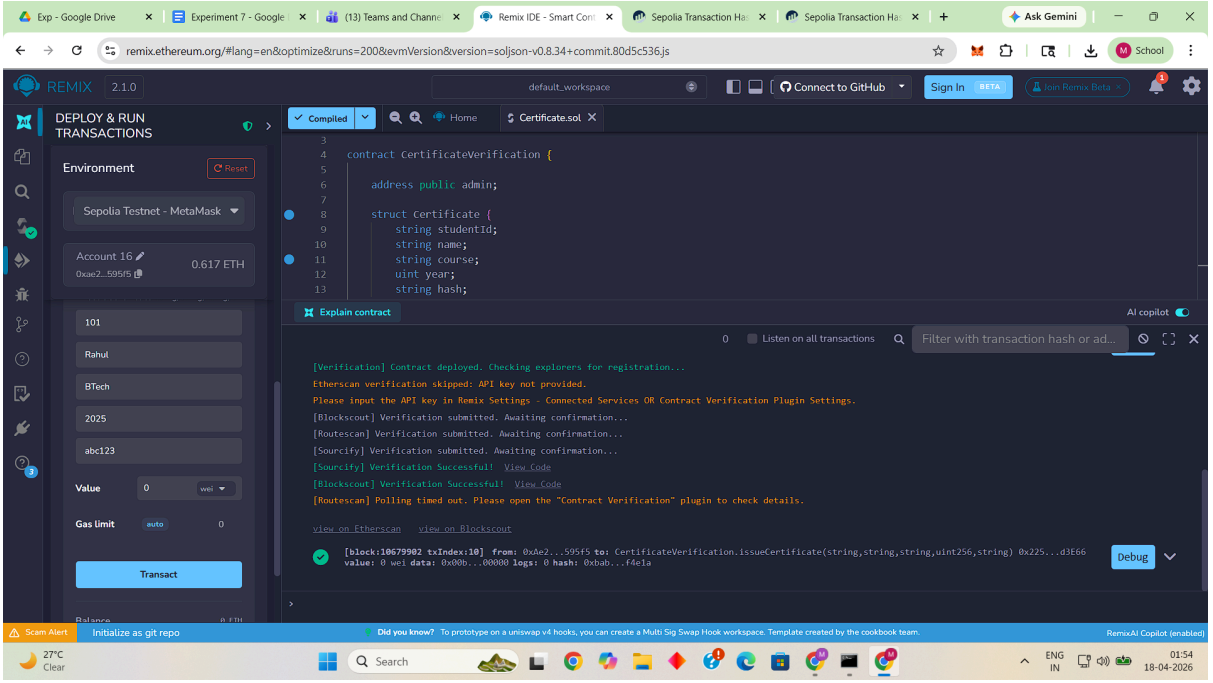

MetaMask
Account 1
Deploy a contract
This site wants you to deploy a contract
Estimated changes
No changes
Network
Sepolia
Request from
remix.ethereum.org
Network fee
$0.68 SepoliaETH
Speed
Market ~12 sec
Cancel
Confirm

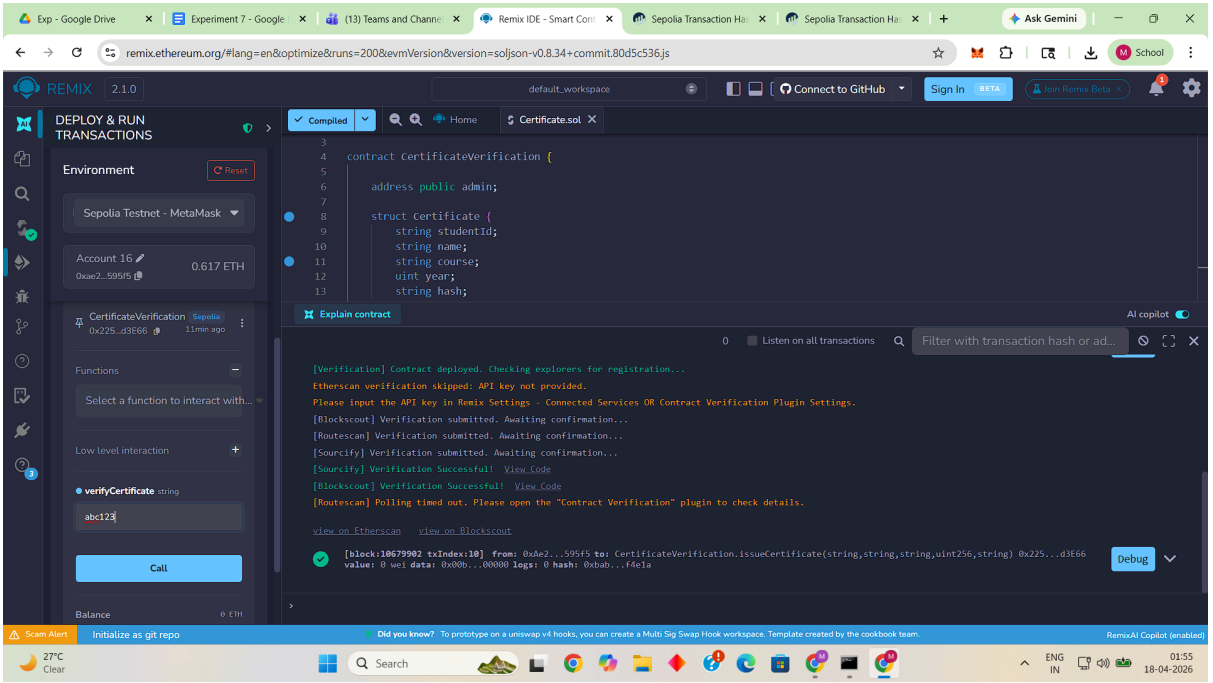

Exp - Google Drive
Experiment 7 - Google Docs
(13) Teams and Channels | Gen
Remix IDE - Smart Contract De
Sepolia Transaction Hash: 0x1
Ask Gemini
sepolia.etherscan.io/tx/0x1364669385a4aa496c75038e95d2dcb3b301e2088962098f1533ce90bdd28de8
Sepolia Testnet
Search by Address / Txn Hash / Block / Token
Etherscan
Home
Blockchain
Tokens
NFTs
More
Transaction Details
Overview
State
API
TRANSACTION ACTION
Call 0x60806040 Method by 0xAe2D048b...4C81595f5
[This is a Sepolia Testnet transaction only]
Transaction Hash: 0x1364669385a4aa496c75038e95d2dcb3b301e2088962098f1533ce90bdd28de8
Status: Success
Block: 10679856 1 Block Confirmation
Timestamp: 22 secs ago (Apr-17-2026 08:13:00 PM +UTC) UTC
From: 0xAe2D048b1A384FEbC3560D9E9cF58694C81595f5
To: [0x225a7d200256e9c76f742170a3193251f4dd3e66 Created]
Value:
Transaction Fee:
This website uses cookies to improve your experience. By continuing to use this website, you agree to its Terms and Privacy Policy.
Got it!
27°C Clear
Search
ENG IN
01:50 18-04-2026

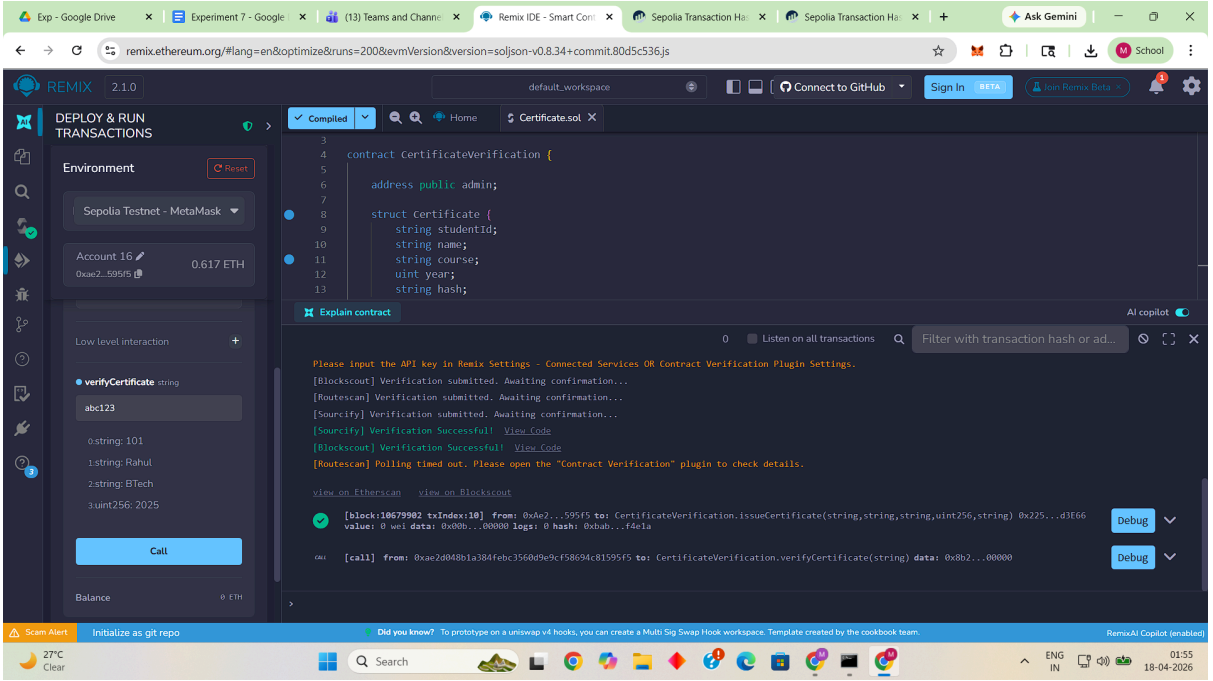

Exp - Google Drive
Experiment 7 - Google Docs
(13) Teams and Channels | Gen
Remix IDE - Smart Contract De
Sepolia Transaction Hash: 0x1
Ask Gemini
remix.ethereum.org/#lang=en&optimize&runs=200&evmVersion&version=soljson-v0.8.34+commit.80d5c536.js
REMIX 2.1.0
default_workspace
Connect to GitHub
Sign In
DEPLOY & RUN TRANSACTIONS
Environment
Reset
Sepolia Testnet - MetaMask
Account 16
0xae2...595f5
0.617 ETH
issueCertificate string, string, string, u...
101
Rahul
BTech
2025
abc123
Value 0 wei
Gas limit auto 0
Transact
Compiled
Home
Certificate.sol
contract CertificateVerification {
address public admin;
struct Certificate {
string studentId;
string name;
string course;
uint year;
string hash;
}
mapping(string => Certificate) public certificates;
constructor() { 852726 gas 827600 gas
admin = msg.sender;
}
modifier onlyAdmin() {
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or ad...
Type the library name to see available commands.
view on Etherscan
view on Blockscout
[block:10679856 txIndex:2] from: 0xAe2...595f5 to: CertificateVerification.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0xe8d...95932
Debug
[Verification] Contract deployed. Checking explorers for registration...
Scam Alert
Initialize as git repo
Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team.
RemixAI Copilot (enabled)
27°C Clear
Search
ENG IN
01:52 18-04-2026


MetaMask
Account 1
Transaction request
Estimated changes
No changes
Network
Sepolia
Request from
remix.ethereum.org
Interacting with
0x225A7...d3E66
Network fee
$0.52 SepoliaETH
Speed
Market ~12 sec
Cancel
Confirm


sepolia.etherscan.io/tx/0xb9fee99bd784591c9642efa6aa17dfa0ee5144f05052f76ebfb9ea597afe6f81
Sepolia Testnet
Search by Address / Txn Hash / Block / Token
TRANSACTION ACTION
Call 0x00b52f29 Method by 0xAe2D048b...4C81595f5 on 0x225A7d20...1f4Dd3E66
[This is a Sepolia Testnet transaction only]
Transaction Hash: 0xb9fee99bd784591c9642efa6aa17dfa0ee5144f05052f76ebfb9ea597afe6f81
Status: Success
Block: 10679902 1 Block Confirmation
Timestamp: 15 secs ago (Apr-17-2026 08:23:12 PM +UTC) UTC
From: 0xAe2D048b1A384FEbC3560D9E9cF58694C81595f5
To: 0x225A7d200256E9C76F742170A3193251f4Dd3E66
Value: 0 ETH
Transaction Fee: 0.000213887573733966 ETH
Gas Price: 1.500102213 Gwei (0.000000001500102213 ETH)
More Details: + Click to show more
This website uses cookies to improve your experience. By continuing to use this website, you agree to its Terms and Privacy Policy. Got it!


remix.ethereum.org/#lang=en&optimize&runs=200&evmVersion&version=soljson-v0.8.34+commit.80d5c536.js
REMIX 2.1.0
DEPLOY & RUN TRANSACTIONS
Environment
Sepolia Testnet - MetaMask
Account 16
0xae2...595f5
0.617 ETH
101
Rahul
BTech
2025
abc123
Value
0
wei
Gas limit
auto
0
Transact
Compiled
Home
Certificate.sol
contract CertificateVerification {
address public admin;
struct Certificate {
string studentId;
string name;
string course;
uint year;
string hash;
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or ad...
[Verification] Contract deployed. Checking explorers for registration...
Etherscan verification skipped: API key not provided.
Please input the API key in Remix Settings - Connected Services OR Contract Verification Plugin Settings.
[Blockscout] Verification submitted. Awaiting confirmation...
[Routescan] Verification submitted. Awaiting confirmation...
[Sourcify] Verification submitted. Awaiting confirmation...
[Sourcify] Verification Successful! View Code
[Blockscout] Verification Successful! View Code
[Routescan] Polling timed out. Please open the "Contract Verification" plugin to check details.
view on Etherscan
view on Blockscout
[block:10679902 txIndex:10] from: 0xAe2...595f5 to: CertificateVerification.issueCertificate(string,string,string,uint256,string) 0x225...d3E66
value: 0 wei data: 0x00b...00000 logs: 0 hash: 0xbab...f4e1a
Debug
Scam Alert
Initialize as git repo
Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team.
RemixAI Copilot (enabled)
27°C Clear
Search
ENG IN
01:54
18-04-2026


remix.ethereum.org/#lang=en&optimize&runs=200&evmVersion&version=soljson-v0.8.34+commit.80d5c536.js
REMIX 2.1.0
DEPLOY & RUN TRANSACTIONS
Environment
Sepolia Testnet - MetaMask
Account 16
0xae2...595f5
0.617 ETH
CertificateVerification Sepolia
0x225...d3E66
11min ago
Functions
Select a function to interact with...
Low level interaction
verifyCertificate string
abc123
Call
Balance
0 ETH
Compiled
Home
Certificate.sol
contract CertificateVerification {
address public admin;
struct Certificate {
string studentId;
string name;
string course;
uint year;
string hash;
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or ad...
[Verification] Contract deployed. Checking explorers for registration...
Etherscan verification skipped: API key not provided.
Please input the API key in Remix Settings - Connected Services OR Contract Verification Plugin Settings.
[Blockscout] Verification submitted. Awaiting confirmation...
[Routescan] Verification submitted. Awaiting confirmation...
[Sourcify] Verification submitted. Awaiting confirmation...
[Sourcify] Verification Successful! View Code
[Blockscout] Verification Successful! View Code
[Routescan] Polling timed out. Please open the "Contract Verification" plugin to check details.
view on Etherscan
view on Blockscout
[block:10679902 txIndex:10] from: 0xAe2...595f5 to: CertificateVerification.issueCertificate(string,string,string,uint256,string) 0x225...d3E66
value: 0 wei data: 0x00b...00000 logs: 0 hash: 0xbab...f4e1a
Debug
Scam Alert
Initialize as git repo
Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team.
RemixAI Copilot (enabled)
27°C Clear
Search
ENG IN
01:55
18-04-2026



<u>1.8 Conclusion</u>: In this Experiment we have learned about web3 also designed and deployed smart contract using Metamask and web3.